Listing 4.10 An example of using the partition key of sensor id, and the sort key of timestamp to efficiently query timeseries data in DynamoDB.

```
import boto3

db = boto3.client('dynamodb', endpoint_url='http://localhost:8000',
                  region_name='us-east-1', aws_access_key_id='dummy',
                  aws_secret_access_key='dummy')#1

response = db.query( #2
   TableName='dtia_sensor_data',
   KeyConditionExpression='sensor_id = :sensor AND #ts BETWEEN
   ➥:start AND :end',
   ExpressionAttributeNames={'#ts': 'timestamp'},
   ExpressionAttributeValues={
       ':sensor': {'S': 'sensor_id1'},
       ':start': {'N': '1749946000000'},
       ':end': {'N': '1749947500000'}
   }
)

print(f"Found {response['Count']} readings:")
for item in response['Items']:
   ts = item['timestamp']['N']
   temp = item['temperature']['N']
   humidity = item['humidity']['N']
   print(f"  {ts}: {temp}°C, {humidity}%")
```

#1 Use a local instance of DynamoDB running in a Docker container in this example.
#2 Query the table based on sensor identifier and start and end time.

4.8.2 Storing object data

For my home digital twin, Amazon S3 serves as the object storage layer for unstructured data including images, documents (appliance manuals, floorplans, maintenance records), and historical sensor data exports as infrequently accessed warm data. S3's key advantages are virtually unlimited storage capacity and multiple storage classes that automatically optimize costs based on access patterns—frequently accessed configuration files remain in standard storage while historical exports transition to cheaper Infrequent Access or Glacier tiers.